1. Introduction of Operating System

An operating system acts as an intermediary between the user of a computer and computer hardware. In short its an interface between computer hardware and user.

It is a program with the help of which we are able to run various applications such as excel, word, paint, calculator, etc.

- The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.
- An operating system is software that manages computer hardware and software. The hardware must provide appropriate mechanisms to ensure the correct operation of the computer system and to prevent user programs from interfering with the proper operation of the system.
- The operating system (OS) is a program that runs at all times on a computer. All other programs, including application programs, run on top of the operating system.
- It does assignment of resources like memory, processors and input / output devices to different processes that need the resources. The assignment of resources has to be fair and secure.
- It mainly acts a government for your system that has different departments to manage different resources.
- Examples are Linux, Unix, Windows 11, MS DOS, Android, macOS and iOS.

2. Fundamental Goals of an Operating System (OS)

Convenience

Goal: Make the computer system easy to use.

- The OS provides a user-friendly interface.
- It allows users to perform tasks like opening files, running programs, and browsing the internet without needing to know complex programming or hardware details.

Security and Protection

Goal: Protect user data and system resources from unauthorized access.

- Prevents one user or program from affecting another.
- Provides login systems, file permissions, and virus protection.

Example: You need a password to log in to your computer or install software.

Control over System Performance

Goal: Monitor and improve the performance of the system.

- The OS keeps track of system health, CPU usage, memory usage, etc.

- It helps optimize the system to avoid slowdowns or crashes.

Example: Task Manager in Windows shows which processes use more CPU or RAM.

Error Detection and Handling
Goal: Detect and fix errors in hardware and software.

- The OS constantly checks for problems and informs the user.
- It can try to correct minor issues automatically.

Example: If a program crashes, the OS can close it safely and show an error message.

Ability to Evolve
Allow the system to adapt to new hardware or technology.
Supports updates and new technology.

3. Operating System services

Program Execution
Loads and runs programs.
Uses CPU scheduling and avoids deadlocks among processes.

File Management
Provides operations: create, delete, read/write files.
Manages permissions and storage on disks/USB.

Memory Management
Allocates/reclaims memory for programs, prevents overlap.
Ensures efficient memory use.

Security & Privacy
Protects the system with login, permissions, firewalls, antivirus.
Ensures user privacy by limiting access to private files.

Resource Management
Allocates CPU time, memory, I/O among processes.
Avoids conflicts and maximizes system performance.

User Interface
Offers CLI (command line) or GUI.
E.g., shell prompts or desktop windows.

Networking
Manages data exchange across devices and internet connections.

Error Handling
Detects hardware/software errors, reports faults, prevents system crashes.

4. Types of Operating Systems:

Multi programming, Time sharing, Real Time, Multithreading, Distributed, Embedded Operating System

4.1 Multi-Programming Operating System

More than one program is present in the main memory and any one of them can be kept in execution. This is used for better utilization of resources.

A **Multi-Programming Operating System** allows **multiple programs** to be loaded into memory **at the same time**, and the CPU executes them **one by one** (not simultaneously, but by switching between them quickly).

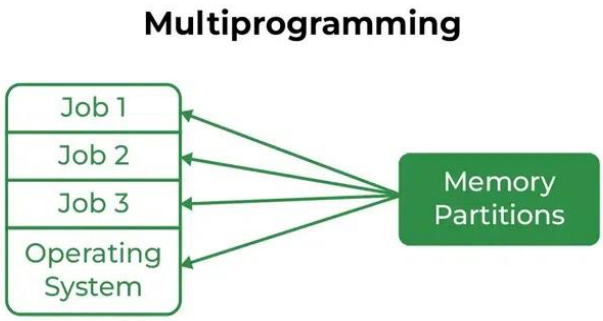
Example: Unix

Why it's Multi-Programming:

- Supports **multiple users** running **multiple programs**.
- If one program is waiting for input/output (I/O), the CPU switches to another program to keep things efficient.
- This maximizes CPU usage and minimizes idle time.

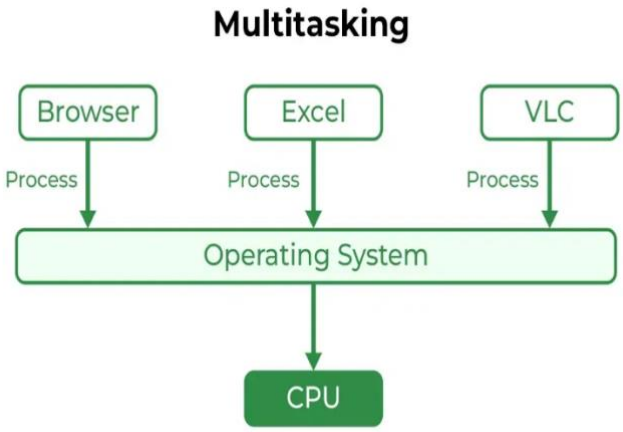
Advantages of, Multi-Programming Operating System

- CPU is better utilized, and the overall performance of the system improves.
- It helps in reducing the response time



4.2 Multi-tasking/Time-sharing Operating systems

It is a type of Multiprogramming system with every process running in round robin manner. Each task is given some time to execute so that all the tasks work smoothly. Each user gets the time of the CPU as they use a single system. These systems are also known as Multitasking Systems. The task can be from a single user or different users. The time that each task gets to execute is called quantum. After this time interval is over, the OS switches over to the next task.



Advantages of Time-Sharing OS

- Each task gets an equal opportunity.
- Fewer chances of duplication of software.
- CPU idle time can be reduced.
- Resource Sharing: Time-sharing systems allow multiple users to share hardware resources such as the CPU, memory, and peripherals, reducing the cost of hardware and increasing efficiency.
- Improved Productivity: Time-sharing allows users to work concurrently, thereby reducing the waiting time for their turn to use the computer. This increased productivity translates to more work getting done in less time.
- Improved User Experience: Time-sharing provides an interactive environment that allows users to communicate with the computer in real time, providing a better user experience than batch processing.

Disadvantages of Time-Sharing OS

- Reliability problem.
- One must take care of the security and integrity of user programs and data.
- Data communication problem.
- High Overhead: Time-sharing systems have a higher overhead than other operating systems due to the need for scheduling, context switching, and other overheads that come with supporting multiple users.
- Complexity: Time-sharing systems are complex and require advanced software to manage multiple users simultaneously. This complexity increases the chance of bugs and errors.

- Security Risks: With multiple users sharing resources, the risk of security breaches increases. Time-sharing systems require careful management of user access, authentication, and authorization to ensure the security of data and software.

4.3 Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.
It is designed to process data and give responses **within a guaranteed time limit**. It is used where **timing is critical**, like in embedded systems, medical devices, robotics, etc.
Example: VxWorks which is used in diverse sectors like aerospace, defense, automotive, medical devices, and industrial automation.
Why it's a Real-Time OS:

- Used in spacecraft, aircraft systems, and industrial robots.
- Ensures that tasks are completed within strict time deadlines.
- Offers predictable and fast response times.

4.4 Multithreading Operating System

A **Multithreading Operating System** allows a **single process** to be **divided into multiple threads**, and these threads can run **concurrently**.
Each **thread** is a smaller part of a program.
Threads **share memory** and resources.
Increases **speed and responsiveness** of applications.

Examples: Windows, Linux, Mac OS

Real-Life Example: When you open a web browser:
One thread loads the page, Another plays a video, Another handles user input.
All work **simultaneously**!

4.5 Distributed Operating System

A **Distributed OS** manages a group of **independent computers** and makes them appear like a **single system** to the user.

Computers are connected via a **network**.
Shares **resources and tasks** among multiple systems.
Improves **efficiency and fault tolerance**.

Example: Google's internal OS

Google search runs on **thousands of machines** working together — the user just sees **one result page**.

4.6 Embedded Operating System

An **Embedded OS** is designed to run on **small, specialized devices** (not general-purpose computers).

- Limited resources: memory, CPU power.
- Performs **specific functions**.
- Often **real-time**.

Examples: FreeRTOS, VxWorks, Embedded Linux

Real-Life Example:

- Washing machines, Smart TVs, Digital cameras, ATM machines

  These devices run **only one task efficiently** using an embedded OS.

5. Operating Systems Structures

Kernel in OS: It is the core part of an operating system. It acts as a bridge between software applications and the hardware of a computer.
The kernel manages system resources, such as the CPU, memory, and devices, ensuring everything works together smoothly and efficiently.
It handles tasks like running programs, accessing files, and connecting to devices like printers and keyboards.

What is a System Structure for an Operating System?
A system structure for an OS is like the blueprint of how an OS is organized and how its different parts interact with each other. Because operating systems have complex structures, we want a structure that is easy to understand so that we can adapt an operating system to meet our specific needs. Similar to how we break down larger problems into smaller, more manageable subproblems, building an operating system in pieces is simpler. The operating system is a component of every segment. The strategy for integrating different operating system components within the kernel can be thought of as an operating system structure.

Types of Operating Systems Structures

- Simple Structure
- Monolithic Structure
- Micro-Kernel Structure
- Layered Structure

Simple structure operating systems do not have well-defined structures and are small, simple, and limited. The interfaces and levels of functionality are not well separated. MS-DOS is an example of such an operating system. In MS-DOS, application programs are able to access the basic I/O routines. These types of operating systems cause the entire system to crash if one of the user programs fails.

Monolithic Structure: It is a type of operating system architecture where the entire operating system is implemented as a single large process in kernel mode. Essential operating system services, such as process management, memory management, file systems, and device drivers, are combined into a single code block.
Advanatages of Monolithic Structure

- Performance of Monolithic structure is fast as since everything runs in a single block, therefore communication between components is quick.
- It is easier to build because all parts are in one code block.

Disadvantages of Monolithic Architecture

- It is hard to maintain as a small error can affect entire system.
- There are also some security risks in the Monolithic architecture**.**

Micro-Kernel Structure
It designs the operating system by removing all non-essential components from the kernel and implementing them as system and user programs. This results in a smaller kernel called the micro-kernel. Advantages of this structure are that all new services need to be added to user space and does not require the kernel to be modified. Thus it is more secure and reliable as if a service fails, then rest of the operating system remains untouched. Mac OS is an example of this type of OS.

Advantages of Micro-kernel Structure

- It makes the operating system portable to various platforms.
- As microkernels are small so these can be tested effectively.

Disadvantages of Micro-kernel Structure

- Increased level of inter module communication degrades system performance.

Layered Structure
An OS can be broken into pieces and retain much more control over the system. In this structure, the OS is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware, and the topmost layer (layer N) is the user interface. These layers are so designed that each layer uses the functions of the lower-level layers. This simplifies the debugging process, if lower-level layers are debugged and an error occurs during debugging, then the error must be on that layer only, as the lower-level layers have already been debugged. The main disadvantage of this structure is that at each layer, the data needs to be modified and passed on which adds overhead to the system. Moreover, careful planning of the layers is necessary, as a layer can use only lower-level layers. UNIX is an example of this structure.

Hybrid-Kernel Structure

Hybrid-Kernel structure is nothing but just a combination of both monolithic-kernel structure and micro-kernel structure. Basically, it combines properties of both monolithic and micro-kernel and make a more advance and helpful approach. It implement speed and design of monolithic and modularity and stability of micro-kernel structure.

Advantages of Hybrid-Kernel Structure

- It offers good performance as it implements the advantages of both structure in it.
- It supports a wide range of hardware and applications.
- It provides better isolation and security by implementing micro-kernel approach.
- It enhances overall system reliability by separating critical functions into micro-kernel for debugging and maintenance.

Disadvantages of Hybrid-Kernel Structure

- It increases overall complexity of system by implementing both structure (monolithic and micro) and making the system difficult to understand.
- The layer of communication between micro-kernel and other component increases time complexity and decreases performance compared to monolithic kernel.

Example: Windows combines features of **monolithic kernels** (fast performance) and **microkernels** (modularity and separation).
The **core (kernel mode)** includes many services like memory management, I/O, and drivers (like a **monolithic kernel**).
But Windows also separates many components and allows user-mode services (like a **microkernel**).

6. Case Study: Linux, Latest Windows Operating System, Mac OS

6.1 Case Study: Linux

Type: Open-source, monolithic kernel

Key Features: Free and open-source, Highly customizable (used in servers, desktops, mobile devices, embedded systems), Supports multi-user, multitasking, multithreading, Secure and stable

Example: Ubuntu

Used In: Web servers (Apache, NGINX), Android smartphones (Linux-based), IoT devices

Advantages: Free to use, Very secure, Ideal for developers

6.2. Case Study: Latest Windows Operating System (Windows 11)

Type: Hybrid Kernel (mix of monolithic and microkernel)

Key Features:

- Modern GUI with taskbar, widgets, and new Start Menu
- Optimized for touchscreen and traditional PCs
- Supports Android apps via Windows Subsystem for Android
- Built-in security with Windows Defender

Used In:

- Personal desktops/laptops, Business environments, Education

Advantages:

- User-friendly interface, Broad software and hardware compatibility, Excellent support for games and applications

Disadvantages:

- Paid license, Heavier system requirements, More vulnerable to malware if not updated

6.3. Case Study: macOS (Latest – macOS Sonoma)

Type: Hybrid Kernel (XNU: part microkernel, part monolithic)

Key Features:

- Sleek, polished graphical interface
- Deep integration with Apple hardware and services
- Based on UNIX (very stable and secure)
- Supports features like Stage Manager, FaceTime handoff, AirDrop

Used In:

- Apple MacBooks, iMacs
- Creative industries (design, music, video editing)

Advantages:

- Highly stable and secure
- Great for multimedia and productivity
- Seamless integration with iPhone/iPad

Disadvantages:

- Expensive hardware
- Limited to Apple devices
- Less flexible/customizable than Linux